

Referensi Buku :

1. Database Systems: Design, Implementation, & Management 14th Edition by Carlos Coronel, Steven Morris – 2022
2. Database System Concepts *Seventh Edition* by Abraham Silberschatz, Henry F. Korth, S. Sudarshan – 2020
3. Connolly, Thomas M & E Begg, Carolyn, *Database System, Apractical Approach to Design, Implementation and Management*, 2015, Pearson Education, United Kingdom
4. Fathansyah, Sistem Basis Data, 2015, Informatika, Bandung
5. Pahlevi, Said Mirza, *Tujuh Langkah Praktis Pembangunan Basis Data*, 2013, Elex Media Komputindo, Jakarta
6. Simarmata, Janner, BasisData, 2010, Andi Offset, Yogyakarta

Pertemuan 4

DESAIN DATABASE LANJUTAN DAN IMPLEMENTASI

Pokok Bahasan :

1. Pengenalan SQL
2. Perintah dalam SQL

Structured Query Language (SQL)

SQL merupakan bahasa query terapan yang banyak digunakan oleh berbagai DBMS, diterapkan dalam berbagai *development tools* dan program aplikasi untuk berinteraksi dengan basis data.

Subdivisi SQL:

1. *Data Definition Language (DDL)*

Query-query ini digunakan untuk mendefinisikan struktur atau skema basis data.

2. *Data Manipulation Language (DML)*

Query-query ini digunakan untuk manajemen data dalam basis data.

PENGELOMPOKAN STATEMEN SQL

1. *Data Definition Language (DDL)*

CREATE DATABASE DROP DATABASE

CREATE TABEL DROP TABEL

CREATE INDEX DROP INDEX

CREATE VIEW DROP VIEW

ALTER TABLE

2. *Data Manipulation Language (DML)*

INSERT, SELECT, UPDATE, DELETE

3. Data Access

GRANT , REVOKE

4. Data Integrity

RECOVER TABLE

5. Auxiliary

SELECT INTO OUTFILE,
LOAD, RENAME TABLE

Data Definition Language (DDL)

A. CREATE

1. Pembuatan Database

Nama Database adalah yang dapat mewakili suatu kejadian dapat berupa nama organisasi atau perusahaan.

Sintaks : `CREATE DATABASE nama_database`

Contoh : Buat database dengan nama KAMPUS
`CREATE DATABASE KAMPUS`

2. Pembuatan Tabel

Sintaks : `CREATE TABLE nama_table
(nama_kolom1 tipe_data_kolom1,
nama_kolom2,tipe_data_kolom2,...)`

Contoh :

Buat struktur tabel dengan nama tabel Mahasiswa dengan data
NIM char(8), NAMA char(25), ALAMAT char(30)

`CREATE TABLE Mahasiswa (NIM char(8) not null,
NAMA char(25) notnull, ALAMAT char(30) notnull)`

DDL lanjutan

3. Pembuatan Index

Sintaks : `CREATE [UNIQUE] INDEX nama_index
ON nama_table (nama_kolom) ;`

Contoh :

Buat index data Mahasiswa berdasarkan NIM dengan nama MHSIDX Dimana NIM tidak boleh sama

```
CREATE UNIQUE INDEX MHSIDX ON Mahasiswa(NIM)
```

4. Pembuatan View

Sintaks :

```
CREATE VIEW nama_view [ (nama_kolom1,...) ]  
AS SELECT statement  
[WITH CHECK OPTION] ;
```

Contoh :

Buat view dengan nama MHSVIEW yang berisi semua data mahasiswa

```
CREATE VIEW MHSVIEW  
AS SELECT * FROM Mahasiswa
```

DDL lanjutan

B. DROP (MENGHAPUS)

1. Menghapus Database

Sintaks : DROP DATABASE nama_db ;

2. Menghapus Tabel

Sintaks : DROP TABLE nama_table ;

3. Menghapus Index

Sintaks : DROP INDEX nama_index ;

4. Menghapus View

Sintaks : DROP VIEW nama_view ;

Contoh :

DROP DATABASE **KAMPUS**;

DROP TABLE MHS;

DROP INDEX MHSIDX;

DROP VIEW MHSVIEW;

DDL lanjutan

C. ALTER TABLE (MERUBAH STRUKTUR TABEL)

Sintaks: ALTER TABLE nama_tabel

ADD nama_kolom jenis_kolom

[FIRST | AFTER nama_kolom]

CHANGE [COLUMN] oldnama newnama

MODIFY nama_kolom jenis_kolom, ...

DROP nama_kolom

RENAME newnama_tabel

Contoh :

1. Tambahkan kolom JKEL dengan panjang 1 char pada tabel Mahasiswa

```
ALTER TABLE Mahasiswa ADD JKEL char(1);
```

2. Ubah panjang kolom JKEL menjadi 15 char

```
ALTER TABLE Mahasiswa MODIFY COLUMN JKEL char(15);
```

3. Hapus kolom JKEL dari data table MHS

```
ALTER TABLE Mahasiswa DROP JKEL;
```

Data Manipulation Language (DML)

A. INSERT

Sintaks SQL yang digunakan untuk penambahan record baru kedalam sebuah tabel.

Sintaks: `INSERT INTO Nama_tabel [(nama_kolom1,...)]
values (nilai atribut1, ...)`

Contoh:Masukan data Mahasiswa dengan Nim 10296832,
Nama Nurhayati beralamat di Jakarta

```
INSERT INTO Mahasiswa (Nim, Nama, Alamat) values  
("10296832","Nurhayati","Jakarta");
```

DML lanjutan

B. UPDATE

Sintaks SQL yang digunakan untuk mengubah nilai atribut pada suatu record dari sebuah tabel.

Sintaks : UPDATE nama_tabel
SET nama_kolom = value_1
WHERE kondisi ;

Contoh:

Ubah alamat menjadi “Depok” untuk mahasiswa yang memiliki NIM “10296832”

```
UPDATE Mahasiswa  
SET ALAMAT="Depok"  
WHERE NIM=" 10296832";
```

DML lanjutan

C. DELETE

Sintaks SQL yang digunakan untuk menghapus record dari sebuah tabel.

Sintaks: DELETE FROM nama_table
WHERE kondisi

Contoh:

Hapus data Mahasiswa yang mempunyai NIM
"21198002"

```
DELETE FROM Mahasiswa  
WHERE NIM=" 21198002"
```

DML lanjutan

Tabel dibawah ini untuk mengerjakan perintah **SELECT**

Tabel Nilai

NIM	KD_MK	MID	FINAL
10296832	KK021	60	75
10296126	KD132	70	90
31296500	KK021	55	40
41296525	KU122	90	80
21196353	KU122	75	75
50095487	KD132	80	0

Tabel Mahasiswa

NIM	NAMA	ALAMAT
10296832	Nurhayati	Jakarta
10296126	Astuti	Jakarta
31296500	Budi	Depok
41296525	Prananigrum	Bogor
50096487	Pipit	Bekasi
21196353	Quraish	Bogor
10296001	Fintri	Depok
21198002	Julizar	Jakarta

Tabel MataKuliah

KD_MK	NAMA_MK	SKS
KK021	Sistem Basis Data	2
KD132	Sistem Informasi Manajemen	3
KU122	Pancasila	2

DML lanjutan

D. SELECT

Sintaks : SELECT [DISTINCT | ALL] nama_kolom
FROM nama_tabel
[WHERE condition]
[GROUP BY column_list]
[HAVING condition]
[ORDER BY column_list [ASC | DESC]]

DML lanjutan

Contoh :

- a. Tampilkan semua data Mahasiswa

```
SELECT NIM,NAMA,ALAMAT FROM Mahasiswa;
```

Atau

```
SELECT * FROM Mahasiswa;
```

Maka hasilnya adalah :

NIM	NAMA	ALAMAT
10296832	Nurhayati	Jakarta
10296126	Astuti	Jakarta
31296500	Budi	Depok
41296525	Prananingrum	Bogor

DML lanjutan

b. Tampilkan Mata Kuliah yang SKS nya 2

```
SELECT NAMA_MK FROM MataKuliah WHERE SKS=2
```

Maka Hasilnya:

NAMA_MK
Sistem Basis Data Pancasila

DML lanjutan

c. Tampilkan semua data nilai dimana nilai MID lebih besar sama dengan 60 atau nilai finalnya lebih besar 75.

maka penulisannya :

```
SELECT * FROM Nilai WHERE MID >= 60 OR FINAL > 75
```

Hasilnya:

NIM	KD_MK	MID	FINAL
10296832	KK021	60	75
10296126	KD132	70	90
41296525	KU122	90	80
21196353	KU122	75	75

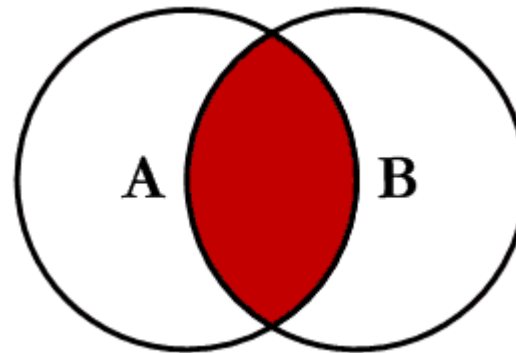
JOIN

JOIN digunakan untuk memilih data dari dua tabel atau lebih.

1. INNER JOIN

Menggabungkan dua tabel dimana diantara dua tabel datanya bersesuaian.

```
SELECT select_list  
FROM Table_A A  
JOIN Table_B B  
ON A.Key = B.Key
```



Source : https://pojokprogrammer.net/id/content/representasi-sql-join-secara-visual?language_content_entity=id

Contoh INNER JOIN

```
SELECT Nilai.NIM, Mahasiswa.NAMA, Nilai.KD_MK, Nilai.MID
FROM Nilai INNER JOIN Mahasiswa
ON Nilai.NIM = Mahasiswa.NIM
```

Hasil :

NIM	NAMA	KD_MK	MID
10296832	Nurhayati	KK021	60
10296126	Astuti	KD132	70
31296500	Budi	KK021	55
41296525	Prananigrum	KU122	90
21196353	Quraish	KU122	75
50095487	Pipit	KD132	80

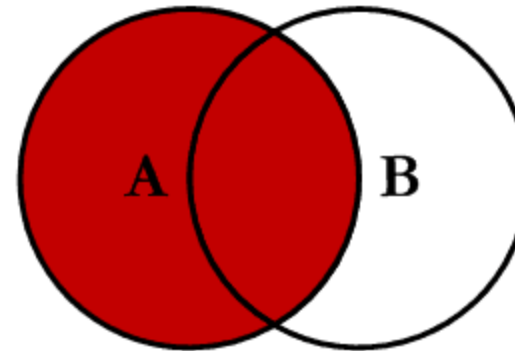
JOIN

JOIN digunakan untuk memilih data dari dua tabel atau lebih.

2. LEFT JOIN atau LEFT OUTER JOIN

Menggabungkan dua tabel dimana diantara dua tabel datanya bersesuaian dan juga semua record pada tabel sebelah kiri.

```
SELECT select_list  
FROM Table_A A  
LEFT JOIN Table_B B  
ON A.Key = B.Key
```



Source : https://pojokprogrammer.net/id/content/representasi-sql-join-secara-visual?language_content_entity=id

Contoh LEFT JOIN

```
SELECT Mahasiswa.NIM, Mahasiswa.NAMA, Nilai.KD_MK,  
        Nilai.MID  
        FROM Mahasiswa LEFT OUTER JOIN Nilai  
        ON Nilai.NIM = Mahasiswa.NIM
```

Hasil:

NIM	NAMA	KD_MK	MID
10296832	Nurhayati	KK021	60
10296126	Astuti	KD132	70
31296500	Budi	KK021	55
41296525	Prananigrum	KU122	90
21196353	Quraish	KU122	75
50095487	Pipit	KD132	80
10296001	Fintri	-	-
21198002	Julizar	-	-

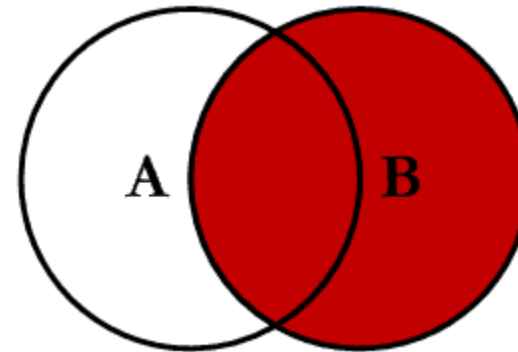
JOIN

JOIN digunakan untuk memilih data dari dua tabel atau lebih.

3. RIGHT JOIN atau RIGHT OUTER JOIN

Menggabungkan dua tabel dimana diantara dua tabel datanya bersesuaian dan juga semua record pada tabel sebelah kanan.

```
SELECT select_list  
FROM Table_A A  
RIGHT JOIN Table_B B  
ON A.Key = B.Key
```



Source : https://pojokprogrammer.net/id/content/representasi-sql-join-secara-visual?language_content_entity=id

Query Terapan lanjutan

Tabel dibawah ini untuk mengerjakan perintah **JOIN**

Tabel Nilai

NIM	KD_MK	MID	FINAL
10296832	KK021	60	75
10296126	KD132	70	90
31296500	KK021	55	40
41296525	KU122	90	80
21196353	KU122	75	75
50095487	KD132	80	0

Tabel Mahasiswa

NIM	NAMA	ALAMAT
10296832	Nurhayati	Jakarta
10296126	Astuti	Jakarta
31296500	Budi	Depok
41296525	Prananigrum	Bogor
50096487	Pipit	Bekasi
21196353	Quraish	Bogor
10296001	Fintri	Depok
21198002	Julizar	Jakarta

Tabel MataKuliah

KD_MK	NAMA_MK	SKS
KK021	Sistem Basis Data	2
KD132	Sistem Informasi Manajemen	3
KU122	Pancasila	2

Contoh RIGHT JOIN

```
SELECT Mahasiswa.NIM, Mahasiswa.NAMA, Nilai.KD_MK,  
Nilai.MID  
FROM Nilai RIGHT OUTER JOIN Mahasiswa  
ON Nilai.NIM = Mahasiswa.NIM
```

Hasil :

NIM	NAMA	KD_MK	MID
10296832	Nurhayati	KK021	60
10296126	Astuti	KD132	70
31296500	Budi	KK021	55
41296525	Prananigrum	KU122	90
21196353	Quraish	KU122	75
50095487	Pipit	KD132	80
10296001	Fintri	-	-
21198002	Julizar	-	-

Contoh Lain Join

tb_kota

id_kota	nama_kota	id_provinsi
1	Jakarta	1
2	Semarang	2
3	Pati	2
4	Bandung	4
5	Surabaya	5
6	Medan	6

tb_provinsi

id_provinsi	nama_provinsi
1	DKI Jakarta
2	Jawa Tengah
3	Papua Barat
4	Jawa Barat
5	Jawa Timur

Tampilkan data-data yang mempunyai nilai sama antara id_provinsi pada table tb_kota dan id_provinsi pada table tb_provinsi

SELECT * FROM tb_kota INNER JOIN tb_provinsi ON tb_kota.id_provinsi = tb_provinsi.id_provinsi;

id_kota	nama_kota	id_provinsi	id_provinsi	nama_provinsi
1	Jakarta	1	1	DKI Jakarta
2	Semarang	2	2	Jawa Tengah
3	Pati	2	2	Jawa Tengah
4	Bandung	4	4	Jawa Barat
5	Surabaya	5	5	Jawa Timur

Tampilkan hanya nama_kota dan nama_provinsi saja

SELECT nama_kota, nama_provinsi FROM tb_kota INNER JOIN tb_provinsi ON tb_kota.id_provinsi = tb_provinsi.id_provinsi;

nama_kota	nama_provinsi
Jakarta	DKI Jakarta
Semarang	Jawa Tengah
Pati	Jawa Tengah
Bandung	Jawa Barat
Surabaya	Jawa Timur

<https://yukcoding.id/belajar-inner-left-right-join-pada-sql/>

Contoh Lain Join

tb_kota

id_kota	nama_kota	id_provinsi
1	Jakarta	1
2	Semarang	2
3	Pati	2
4	Bandung	4
5	Surabaya	5
6	Medan	6

tb_provinsi

id_provinsi	nama_provinsi
1	DKI Jakarta
2	Jawa Tengah
3	Papua Barat
4	Jawa Barat
5	Jawa Timur

Tampilkan data-data Pada table *tb_provinsi* (kanan).

```
SELECT * FROM tb_kota LEFT JOIN tb_provinsi ON  
tb_kota.id_provinsi = tb_provinsi.id_provinsi;
```

id_kota	nama_kota	id_provinsi	id_provinsi	nama_provinsi
1	Jakarta	1	1	DKI Jakarta
2	Semarang	2	2	Jawa Tengah
3	Pati	2	2	Jawa Tengah
4	Bandung	4	4	Jawa Barat
5	Surabaya	5	5	Jawa Timur
6	Medan	6	NULL	NULL

<https://yukcoding.id/belajar-inner-left-right-join-pada-sql/>

Contoh Lain Join

tb_kota

id_kota	nama_kota	id_provinsi
1	Jakarta	1
2	Semarang	2
3	Pati	2
4	Bandung	4
5	Surabaya	5
6	Medan	6

tb_provinsi

id_provinsi	nama_provinsi
1	DKI Jakarta
2	Jawa Tengah
3	Papua Barat
4	Jawa Barat
5	Jawa Timur

```
SELECT * FROM tb_kota RIGHT JOIN tb_provinsi ON  
tb_kota.id_provinsi = tb_provinsi.id_provinsi;
```

id_kota	nama_kota	id_provinsi	id_provinsi	nama_provinsi
1	Jakarta	1	1	DKI Jakarta
2	Semarang	2	2	Jawa Tengah
3	Pati	2	2	Jawa Tengah
4	Bandung	4	4	Jawa Barat
5	Surabaya	5	5	Jawa Timur
NULL	NULL	NULL	3	Papua Barat

<https://yukcoding.id/belajar-inner-left-right-join-pada-sql/>

Data Access

1. GRANT → Memberikan hak akses / hak istimewa pengguna

Sintaks : GRANT hak_akses ON nama_db
 TO nama_pemakai
 [IDENTIFIED BY] [PASSWORD] 'Password'
 [WITH GRANT OPTION] ;

GRANT hak_akses ON [nama_db]nama_tabel
TO nama_pemakai
[IDENTIFIED BY] [PASSWORD] 'Password'
[WITH GRANT OPTION];

Contoh :

Berikan hak akses kepada Adi untuk menampilkan
nilai final test pada tabel Nilai.

GRANT SELECT (FINAL) ON NILAI TO ADI

Data Access lanjutan

2. **REVOKE** → Menarik hak akses pengguna yang diberikan lewat perintah GRANT

Sintaks : REVOKE hak_akses ON nama_db
FROM nama_pemakai ;

REVOKE hak_akses ON nama_tabel
FROM nama_pemakai ;

Contoh :

Tarik kembali dari Adi hak akses untuk menampilkan nilai final test

REVOKE SELECT (FINAL) ON NILAI FROM ADI

Data Integrity

RECOVER TABLE

Sintaks : `RECOVER TABLE nama_tabel`

Contoh :

Kembalikan keadaan data mahasiswa seperti pada saat sebelum terjadi kerusakan

```
RECOVER TABLE MHS ;
```

Auxiliary

1. **SELECT ... INTO OUTFILE** 'filename'

Sintaks ini digunakan untuk mengekspor data dari tabel ke file lain.

Sintaks : SELECT ... INTO
 OUTFILE 'Nama File'
 [FIELDS | COLUMNS]
 [TERMINATED BY '*string*']
 [[OPTIONALLY] ENCLOSED BY '*char*']
 [ESCAPED BY '*char*']]

Contoh :

Ubah semua data mahasiswa ke bentuk ASCII dan disimpan ke file teks di directory/home/adi dengan pemisah antar kolom '|'

```
SELECT * FROM MHS  
INTO OUTFILE "/home/adi/teks"  
FIELDS TERMINATED BY "|";
```

Auxiliary lanjutan

2. LOAD

Sintaks query ini digunakan untuk mengimpor data dari file lain ke tabel.

```
Sintaks :  LOAD DATA INFILE " nama_path"
           INTO TABLE nama_tabel [ nama_kolom] ;
           [FIELDS | COLUMNS]
           [TERMINATED BY 'string']
           [[OPTIONALLY] ENCLOSED BY 'char']
           [ESCAPED BY 'char'] ]
```

Contoh :

Memasukkan data-data dari file teks yang berada pada direktori “/home/adi” ke dalam tabel MHS_2. Dimana pemisah antara kolom dalam file teks adalah tab (\t) :

```
LOAD FROM “/home/adi/teks”
INTO MHS_2
FIELDS TERMINATED BY ‘\t’;
```

Auxiliary lanjutan

3. RENAME TABLE

Sintaks :

```
RENAME TABLE OldnamaTabel  
TO NewNamaTabel
```

Contoh :

```
RENAME TABLE MHS  
TO MAHASISWA
```

Fungsi Aggregate

MENGGUNAKAN FUNGSI AGGREGATE :

1. COUNT digunakan untuk menghitung jumlah.
Menghitung jumlah record mahasiswa dari tabel MAHASISWA

```
SELECT COUNT(*) FROM MAHASISWA
```
2. SUM digunakan untuk menghitung total dari kolom yang mempunyai tipe data numerik.

```
SELECT SUM(SKS) AS 'TOTAL SKS' FROM MATAKULIAH
```

Fungsi Aggregate lanjutan

3. AVG digunakan untuk menghitung rata-rata dari data-data dalam sebuah kolom.

```
SELECT AVG(FINAL) AS 'FINAL' FROM Nilai
```

4. MIN digunakan untuk menghitung nilai minimal dalam sebuah kolom.

```
SELECT MIN(FINAL) FROM Nilai
```

5. MAX digunakan untuk menghitung nilai maksimum dalam sebuah kolom

```
SELECT MAX(MID) FROM Nilai
```

SUBQUERY

SUBQUERY

Adalah subselect yang dapat digunakan di klausa WHERE dan HAVING dipernyataan select luar untuk menghasilkan tabel akhir.

Aturan-aturan untuk membuat subquery, yaitu :

1. Klausa Order By tidak boleh digunakan di subquery, Order By hanya dapat digunakan di pernyataan Select luar.
2. Klausa subquery Select harus berisi satu nama kolom tunggal atau ekspresi kecuali untuk subquery-subquery menggunakan kata kunci EXIST
3. Secara default nama kolom di subquery mengacu ke nama tabel di klausa FROM dari subquery tersebut.
4. Saat subquery adalah salah satu dua operan dilibatkan di perbandingan, subquery harus muncul disisi kanan perbandingan

Subquery lanjutan

Penggunaanan ANY dan ALL

Jika subquery diawali kata kunci ALL, syarat hanya akan bernilai TRUE jika dipenuhi semua nilai yang dihasilkan subquery itu.

Jika subquery diawali kata kunci ANY, syaratnya akan bernilai TRUE jika dipenuhi sedikitnya satu nilai yang dihasilkan subquery tersebut.

Subquery lanjutan

Penggunaanan **EXIST** DAN **NOT EXIST**

EXIST akan mengirim nilai TRUE jika dan hanya jika terdapat sedikitnya satu baris di tabel hasil yang dikirim oleh subquery dan EXIST mengirim nilai FALSE jika subquery mengirim tabel kosong.

Untuk NOT EXIST kebalikan dari EXIST.

(Masing-masing dosen membuat contoh untuk subquery)

Subquery lanjutan

CONTOH SUBQUERY :

1. Ambil nilai mid dan final dari mahasiswa yang bernama Astuti.

```
SELECT MID, FINAL FROM NILAI WHERE NIM=(  
SELECT NIM FROM MAHASISWA WHERE  
NAMA='Astuti')
```

2. Ambil nilai kode matakuliah, mid dan final dari mahasiswa yang tinggal di jakarta.

```
SELECT KD_MK, MID, FINAL FROM NILAI WHERE NIM  
IN(SELECT NIM FROM MAHASISWA WHERE ALAMAT =  
'Jakarta')
```

Subquery lanjutan

3. Ambil nama-nama mahasiswa yang mengikuti ujian.

```
SELECT NAMA FROM MAHASISWA WHERE EXISTS  
(SELECT NIM FROM NILAI WHERE NILAI.NIM=  
MAHASISWA.NIM)
```

4. Ambil nama-nama mahasiswa yang tidak mengikuti ujian.

```
SELECT NAMA FROM MAHASISWA WHERE NOT  
EXISTS (SELECT NIM FROM NILAI WHERE NILAI.NIM=  
MAHASISWA.NIM)
```

Aplikasi yang digunakan sebagai contoh adalah Mysql

Dari Address ketik : <http://localhost/phpmyadmin>

Tampilan user ketik **root** dan **password** dikosongkan

Latihan Menggunakan SQL :

Tabel Nilai

NIM	KD_MK	MID	FINAL
10296832	KK021	60	75
10296126	KD132	70	90
31296500	KK021	55	40
41296525	KU122	90	80
21196353	KU122	75	75
50095487	KD132	80	0

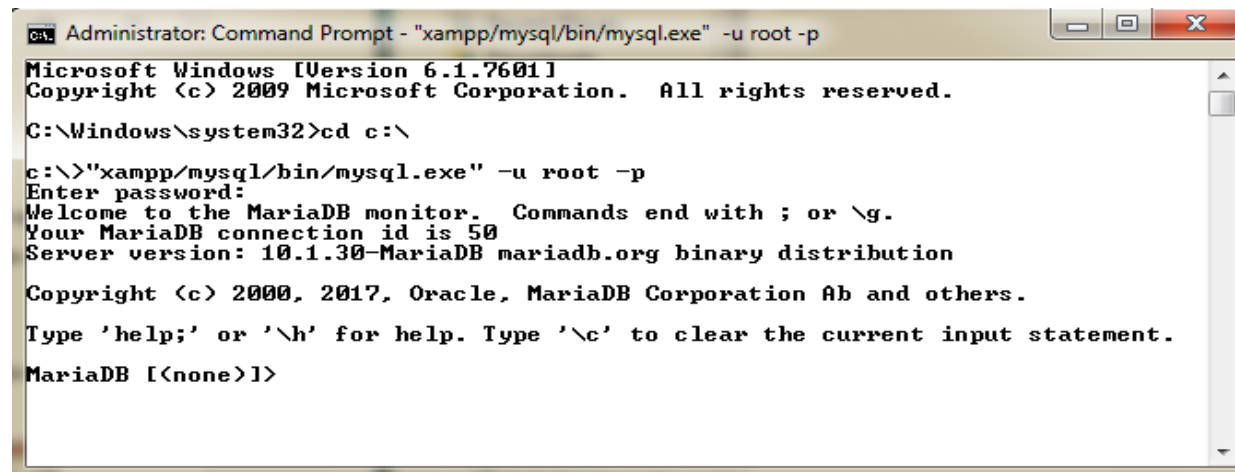
Tabel Mahasiswa

NIM	NAMA	ALAMAT
10296832	Nurhayati	Jakarta
10296126	Astuti	Jakarta
31296500	Budi	Depok
41296525	Prananigrum	Bogor
50096487	Pipit	Bekasi
21196353	Quraish	Bogor
10296001	<u>Fintri</u>	<u>Depok</u>
21198002	<u>Julizar</u>	Jakarta

Tabel MataKuliah

KD_MK	NAMA_MK	SKS
KK021	Sistem Basis Data	2
KD132	Sistem Informasi Manajemen	3
KU122	Pancasila	2

1. Pilih menu *Start - All Programs - Accessories* Kemudian Pilih *Command Prompt*.
2. Klik Kanan pada command prompt atau cmd, lalu pilih *Run as administrator*.
3. Ketikan perintah `cd c:\` (lalu kemudian tekan Enter)
4. Lalu ketikan perintah `"xampp/mysql/bin/mysql.exe" -u root -p` (lalu kemudian tekan Enter)



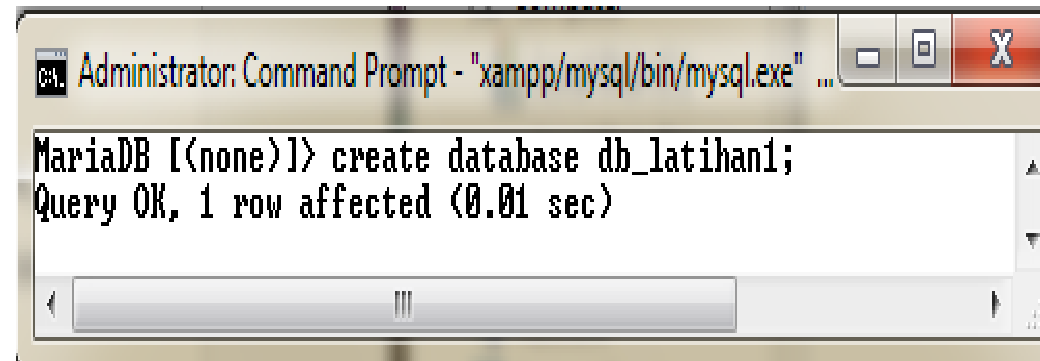
```
Administrator: Command Prompt - "xampp/mysql/bin/mysql.exe" -u root -p
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Windows\system32>cd c:\

c:\>"xampp/mysql/bin/mysql.exe" -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 50
Server version: 10.1.30-MariaDB mariadb.org binary distribution
Copyright (c) 2000, 2017, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
MariaDB [(none)]>
```

Membuat Database dan Tabel Sederhana

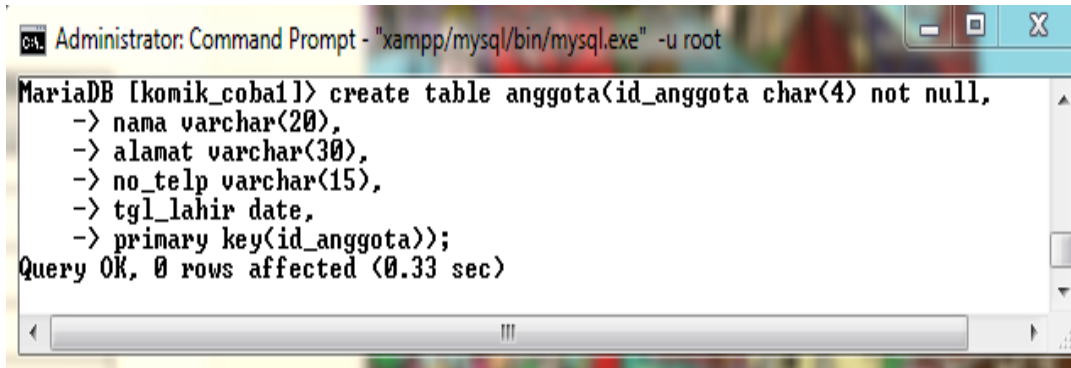
- Setelah masuk MariaDB, ketikkan perintah berikut: `create database db_latihan1;`
- Lihat hasilnya dengan perintah ***show databases;***
- Lalu, untuk membuat tabel terlebih dahulu kita aktifkan database db_latihan1 dengan perintah ***use.***



```
Administrator: Command Prompt - "xampp/mysql/bin/mysql.exe" ...  
MariaDB [(none)]> create database db_latihan1;  
Query OK, 1 row affected (0.01 sec)
```

Membuat Database dan Tabel Sederhana

- Perintah untuk membuat tabel, yaitu dengan perintah *create table nama_tabel (spesifikasi tabel);*



```
Administrator: Command Prompt - "xampp/mysql/bin/mysql.exe" -u root
MariaDB [komik_coba11]> create table anggota(id_anggota char(4) not null,
-> nama varchar(20),
-> alamat varchar(30),
-> no_telp varchar(15),
-> tgl_lahir date,
-> primary key(id_anggota));
Query OK, 0 rows affected (0.33 sec)
```

- Untuk melihat daftar tabel pada suatu database, gunakan perintah *show tables;*
- Lanjutkan untuk membuat seluruh table yang ada.

Tugas Kelompok :

1. Berdasarkan sistem usulan buat ER Modeling dan Normalisasi-nya, implementasikan database tersebut menggunakan Bahasa SQL.
2. Tugas ini di laporkan ke pengampu mata kuliah di pertemuan 5.